

Výjimky označují nějaký výjimečný/neočekávaný stav v programu. Např. „soubor na zadané cestě nenalezen“, „internetové spojení přerušeno“ nebo „zadanou hodnotu nelze převést na číslo“.

Základní třídou pro všechny výjimky je *BaseException* (v Javě *Exception*) – označují „systémové“ chyby, které mají vynutit ukončení celého programu. Výjimky nižší závažnosti dědí z třídy *Exception* (v Javě vhodnější název *RuntimeException* – „výjimka za běhu“), které označují očekávaný chybový stav za běhu programu (zpravidla závislé na externích vlivech – data v souboru nebo vstup uživatele).

U výjimek *Exception* (za běhu) se očekává, že je vývojář zachytí a mimořádný stav, který nastal, náležitě vyřeší. Výjimky *BaseException* se mohou také zachytávat, ale bývají to neočekávané stavy, tudíž není ani obvyklé je v programu řešit.

Na rozdíl od Javy spadají v Pythonu pod *Exception* i chyby v zápisu programu – *SyntaxError* a podobné („za *while* chybí dvojtečka“, „nepravidelné odsazení kódu“ apod.). V Javě tyto chyby nejsou vůbec výjimky, protože se validita programu kontroluje hned při kompilaci, a ne až za běhu.

Pod *BaseException* patří např. *SystemExit* (volání *exit()*) a *KeyboardInterrupt* (*Ctrl+C* v konzoli), což jsou obojí stavy, kdy se má vynutit ukončení aplikace.

Pod *Exception* patří např. *AttributeError* (atribut v objektu nenalezen), *ZeroDivisionError* (dělení nulou) a již zmíněný *SyntaxError*.

Výjimky se vyvolávají příkazem *raise*:

Pokud nechceme zadat parametry konstruktoru výjimky, nemusíme ani uvádět závorky.

```
raise ValueError
raise ValueError()
raise ValueError("Hodnota musí být větší než 0")
```

Řídící strukturou pro zachytávání a zpracovávání výjimek je tzv. *try-except* blok. *Try* blok obaluje kód, u kterého očekáváme, že nastane nějaká výjimka. Každý *try* blok musí mít alespoň jeden *except* blok (v Javě *catch* blok). Každý *except* blok má určený filtr na výjimky, které chce zpracovávat.

```
while True:
    try:
        cislo: int = int(input("Zadej číslo: "))
        break
    except ValueError:
        print("Neplatné číslo.")
```

Po vyvolání výjimky kdekoliv v kódu se hledá první obalující *try-except* blok i přes nadřazené volající funkce. Jakmile se na daný blok narazí, zkusí se typ výjimky zachytit v uvedených *except* blocích. Pokud filtr sedí, výjimka se zachytí a v *except* bloku se zpracuje. Pokud žádný filtr neseď, výjimka pokračuje v hledání dále v nadřazených funkcích:

```
def vnitřni_funkce(hodnota: int) -> int:
    if hodnota <= 0:
        raise ValueError("Hodnota musí být větší než 0")
    return hodnota * 2

def vnější_funkce(hodnota: int) -> None:
    vnitřni_funkce(hodnota)
    vnitřni_funkce(hodnota - 1)
    vnitřni_funkce(hodnota + 1)

try:
    vnější_funkce(1)
except ValueError:
    print("Špatná hodnota!")
```

Pokud se nenalezne žádný pasující filtr v celém tzv. *call stacku*, program se ukončí s **chybovou hláškou dané výjimky** a výpisem *call stacku* – tzv. „*traceback*“:

*Traceback (most recent call last):*

*File "c:\Users\user\main.py", line 46, in <module>*

*vnější\_funkce(1)*

*File "c:\Users\user\main.py", line 43, in vnější\_funkce*

*vnitřni\_funkce(hodnota - 1)*

*File "c:\Users\user\main.py", line 38, in vnitřni\_funkce*

*raise ValueError("Hodnota musí být větší než 0")*

***ValueError: Hodnota musí být větší než 0***

*Except* bloků může být za *try* blokem více:

```
while True:
    try:
        cislo: int = int(input("Zadej číslo: "))
        break
    except KeyboardInterrupt:
        print("Nezadaná hodnota, nastavuji 0...")
        cislo = 0
        break
    except ValueError:
        print("Neplatné číslo.")
```

Typy výjimek se kontrolují ve stejném pořadí, jako ve kterém jsou zapsány. Což je důležité kvůli dědičnosti výjimek. *ValueError* je potomek *Exception*, ale zpracuje se ve své větvi. Kdyby byly *except* bloky prohozené, *ValueError* větev by se nikdy nemohla provést:

```
while True:
    try:
        cislo: int = int(input("Zadej číslo: "))
        break
    except ValueError:
        print("Neplatné číslo.") # <--- správně
    except Exception:
        print("Neznámá chyba")
```

```
while True:
    try:
        cislo: int = int(input("Zadej číslo: "))
        break
    except Exception:
        print("Neznámá chyba") # <--- "chybně" - ValueError se zachytí zde
    except ValueError:
        print("Neplatné číslo.")
```

Navíc je možné získat instanci zachycené výjimky a zpracovat ji – upravit jí data nebo ji opět vyvolat („poslat dál“).

```
while True:
    try:
        cislo: int = int(input("Zadej číslo: "))
        break
    except KeyboardInterrupt:
        print("Vynucené ukončení...")
        cislo = 0
        break
    except ValueError:
        print("Neplatné číslo.")
    except Exception as exc:          # zachycení instance
        print("Neznámá chyba")
        raise exc                    # "poslání dál"
```

Except blok může zachytávat více typů výjimek naráz:

```
try:
    delitel: int = int(input("Zadej dělitele: "))
    vysledek: float = 100 / delitel
except (ZeroDivisionError, ValueError):  # zadáno jako tuple - závorky
    print("Chyba dělení...")
    vysledek = 0
```

Vlastní výjimky se vytváří děděním z třídy *Exception*, popř. *BaseException* anebo jiných existujících výjimek:

```
class MojeVyjimka(Exception):
    pass

raise MojeVyjimka
```

*Try-except* blok lze ještě rozšířit o jeden *finally* blok – vznikne tak tzv. *try-except-finally* blok.

*Finally* blok se provede **vždy** po *try* bloku. A to, i když se vyvolá výjimka:

```
pokusy = 0
while True:
    try:
        cislo: int = int(input("Zadej číslo: "))
        break
    except ValueError:
        print("Neplatné číslo.")
    finally:
        pokusy += 1
print("Počet pokusů:", pokusy)
```

Nejdříve se zpracovává *try* blok. Pokud se uvnitř něj vyvolá výjimka, vykoná se *finally* blok a až teprve poté se začnou vykonávat *except* bloky.

Toto chování je užitečné převážně při práci se soubory nebo při komunikaci po síti, kde je většinou nutné soubor nebo komunikaci řádně ukončit, jinak mohou zdroje zůstat zablokované.

Nepoužívejte výjimky na často opakované části kódu, kde mohou výjimky nastávat často. Většinou se v těchto situacích dá vyvolání výjimky snadno zabránit vhodnou podmínkou. Vyvolání a zachycení výjimky je totiž oproti podmínce řádově pomalejší operace.

Např. u převodu řetězce na číslo lze použít „regex match“ – kontrolu, zda řetězec obsahuje jen znaky číslic. Nebo u dělení podmínkou kontrolovat, zda dělitel není 0.

V opačném případě je v některých situacích výjimka levnější než miliarda testování podmínek. Vždy záleží na situaci a běhovém prostředí.